Phase 6 — Notebook 1: Data Validation and Monitoring

This notebook demonstrates the input validation layer that guards both prediction endpoints. It also inspects the existing prediction audit log to verify governance compliance.

Scope:

- Load and inspect the enriched feature schemas
- Run `validate_payload` against valid and invalid examples for both models
- Show all three check types in action: missing values, numeric range violations, unseen categories
- Parse and summarise the live audit log from Phase 5

In [ ]:
```
import json
import re
import sys
from pathlib import Path

import pandas as pd

# Resolve paths relative to the notebook location
PHASE6_DIR = Path(".").resolve()
REPO_ROOT = PHASE6_DIR.parent.parent
MODEL_FILES = REPO_ROOT / "Model Files"
LOG_PATH = REPO_ROOT / "Notebooks" / "Phase 5 — Deployment and API Integration" / "logs" / "predictions.log"

# Make validate.py importable
if str(PHASE6_DIR) not in sys.path:
    sys.path.insert(0, str(PHASE6_DIR))
from validate import validate_payload

print("validate_payload imported successfully")
```

1. Feature Schema Overview

In [ ]:
```
with open(MODEL_FILES / "patient_risk_model_feature_schema.json") as f:
    risk_schema = json.load(f)

with open(MODEL_FILES / "claim_outcome_model_feature_schema.json") as f:
    claim_schema = json.load(f)

def schema_summary(schema: dict) -> pd.DataFrame:
    rows = []
    for feat in schema["features"]:
        c = feat.get("constraints", {})
        rows.append({
            "Feature": feat["name"],
            "Type": feat["type"],
            "Required": feat.get("required", False),
            "min_value": c.get("min_value", ""),
            "max_value": c.get("max_value", ""),
            "allowed_values": ", ".join(str(v) for v in c["allowed_values"]) if "allowed_values" in c else "",
```

```
        })
    return pd.DataFrame(rows)

print(f"=== Patient Risk Schema (v{risk_schema['version']}) ===")
display(schema_summary(risk_schema))

print(f"\n=== Claim Outcome Schema (v{claim_schema['version']}) ===")
display(schema_summary(claim_schema))
```

2. Validation Demo — Patient Risk Model

2a. Valid payload (should pass)

In [ ]:
```
valid_risk_payload = {
    "chronic_flag": 1,
    "gender": "F",
    "visit_frequency": 5,
}

ok, errors = validate_payload(valid_risk_payload, risk_schema)
print(f"Valid: {ok}")
print(f"Errors: {errors}")
```

2b. Check 1 — Missing required field

In [ ]:
```
missing_field_payload = {
    "chronic_flag": 1,
    # gender is missing
    "visit_frequency": 3,
}

ok, errors = validate_payload(missing_field_payload, risk_schema)
print(f"Valid: {ok}")
for e in errors:
    print(f"  ✗ {e}")
```

2c. Check 2 — Numeric range violation

In [ ]:
```
range_violation_payload = {
    "chronic_flag": 1,
    "gender": "M",
    "visit_frequency": 0,   # must be >= 1
}

ok, errors = validate_payload(range_violation_payload, risk_schema)
print(f"Valid: {ok}")
for e in errors:
    print(f"  ✗ {e}")
```

2d. Check 3 — Unseen category

In [ ]:
```
unseen_category_payload = {
    "chronic_flag": 1,
    "gender": "X",           # not in ["M", "F"]
    "visit_frequency": 3,
}

ok, errors = validate_payload(unseen_category_payload, risk_schema)
print(f"Valid: {ok}")
for e in errors:
    print(f"  ✗ {e}")
```

3. Validation Demo — Claim Outcome Model

3a. Valid payload

In [ ]:
```
valid_claim_payload = {
    "age": 45,
    "gender": "M",
    "city": "Mumbai",
    "insurance_provider": "HealthPlus",
    "chronic_flag": 0,
    "department": "General",
    "visit_type": "OPD",
    "length_of_stay_hours": 4.5,
    "billed_amount": 12000.0,
    "payment_days": 14.0,
    "visit_frequency": 3,
    "lag_time": 5,
}

ok, errors = validate_payload(valid_claim_payload, claim_schema)
print(f"Valid: {ok}")
print(f"Errors: {errors}")
```

3b. Multiple violations at once

In [ ]:
```
multi_violation_payload = {
    "age": 200,                        # exceeds max_value=120
    "gender": "M",
    "city": "Singapore",               # unseen category
    "insurance_provider": "HealthPlus",
    "chronic_flag": 0,
    "department": "General",
    "visit_type": "OPD",
    "length_of_stay_hours": -1.0,      # below min_value=0
    # billed_amount is missing (required)
    "payment_days": 14.0,
    "visit_frequency": 3,
    "lag_time": 5,
}

ok, errors = validate_payload(multi_violation_payload, claim_schema)
print(f"Valid: {ok}")
```

```
for e in errors:
    print(f"  ✗ {e}")
```

4. Validation Rules Summary Table

In [ ]:
```
all_rules = []
for model_name, schema in [("patient_risk", risk_schema), ("claim_outcome", claim_schema)]:
    for feat in schema["features"]:
        c = feat.get("constraints", {})
        checks = []
        if feat.get("required"):
            checks.append("not-null")
        if "min_value" in c or "max_value" in c:
            bounds = []
            if "min_value" in c:
                bounds.append(f">= {c['min_value']}")
            if "max_value" in c:
                bounds.append(f"<= {c['max_value']}")
            checks.append("range: " + ", ".join(bounds))
        if "allowed_values" in c:
            checks.append(f"in {c['allowed_values']}")
        all_rules.append({
            "Model": model_name,
            "Feature": feat["name"],
            "Type": feat["type"],
            "Validation Rules": " | ".join(checks) if checks else "type check only",
        })

rules_df = pd.DataFrame(all_rules)
display(rules_df)
```

5. Prediction Audit Log Inspection

The API logs every prediction to `Phase 5/logs/predictions.log`. Each line contains:

- `prediction_id` — UUID for traceability
- `model` + version — model identity
- `ts` — UTC ISO-8601 timestamp
- `hash` — SHA-256 of input features (privacy-preserving)
- `label` — predicted class
- `score` — model confidence

In [ ]:
```
if LOG_PATH.exists():
    log_lines = LOG_PATH.read_text().strip().splitlines()
    prediction_lines = [l for l in log_lines if "PREDICTION" in l]
    print(f"Total prediction log entries: {len(prediction_lines)}")
    print("\nSample entries (last 5):")
    for line in prediction_lines[-5:]:
        print(" ", line)
else:
```

```
    print(f"Log file not found at: {LOG_PATH}")
    print("Start the API server and make a prediction first.")
```

In [ ]:
```
if LOG_PATH.exists() and prediction_lines:
    pattern = re.compile(
        r"PREDICTION \| id=(?P<id>[\w-]+) \| model=(?P<model>[\w_]+) v(?P<version>[\d.]+) "
        r"\| ts=(?P<ts>[\S]+) \| hash=(?P<hash>\w+) \| label=(?P<label>\w+) \| score=(?P<score>[\d.]+)"
    )
    records = []
    for line in prediction_lines:
        m = pattern.search(line)
        if m:
            records.append(m.groupdict())

    log_df = pd.DataFrame(records)
    log_df["score"] = log_df["score"].astype(float)
    log_df["ts"] = pd.to_datetime(log_df["ts"])

    print("Parsed audit log:")
    display(log_df.head(10))

    print("\nPrediction distribution by model and label:")
    display(log_df.groupby(["model", "label"]).size().reset_index(name="count"))

    print("\nAverage confidence by model:")
    display(log_df.groupby("model")["score"].mean().reset_index())
```

Summary

Check Type	Scope	Behaviour
Missing / null	All required features	HTTP 422 with field name
Numeric range	`int8` , `int32` , `float32` features with bounds	HTTP 422 with value and bounds
Unseen category	All `category` and `int8` flag features with `allowed_values`	HTTP 422 with value and allowed set

All violations are collected before responding, so a single bad request surfaces **all** errors at once. The audit log captures every successful prediction with a SHA-256 feature hash, enabling traceability without storing raw PII.